

Product-to-Production Guide for Product Managers

Introduction

This guide maps the typical work a developer and their organization complete after purchasing a product, putting it into production, and operating it through retirement. Product managers can use the journey to identify friction, dependencies, and manual work that the product could eventually automate, simplify, validate, or eliminate. Replace the example steps with evidence from your customers' actual workflows.

Definitions

In this guide, “your product” means the platform being evaluated by the product manager. The placeholder “yourproduct” appears only in example commands, URLs, and API paths. Named tools and services—such as Terraform, Ansible, GitHub Actions, GitLab CI, Slack, Microsoft Teams, and PagerDuty—are third-party developer or operational tools.

For each activity, identify whether the customer, the product team, or a shared operational team owns the work. Then document what the product provides, what the customer must supply, where approval is required, and who resolves exceptions. This distinction prevents customer prerequisites from being mistaken for product capabilities and exposes handoffs that the Golden Path should simplify.

Developer Tools

Developers usually interact with platforms through three related interfaces.

- **Infrastructure as Code (IaC):** describes the desired environment in configuration so it can be provisioned and changed consistently.
- **Application programming interface (API):** lets software call product capabilities directly for automation and custom integrations.
- **Command-line interface (CLI):** lets a developer or pipeline run product operations through commands.

Supporting all three can require shared platform capabilities plus separate documentation, authentication, packaging, testing, versioning, and support work for each interface.

A CI/CD pipeline is a repeatable, automated sequence of steps that validates, tests, approves, deploys, and verifies a software or infrastructure change.

The pipeline is the execution environment; its steps may apply IaC, call APIs, or run CLI commands.

- IaC is commonly the primary method for repeatable provisioning and lifecycle changes,
- APIs support custom integrations or capabilities not covered by IaC, and
- CLIs are often used for simpler scripted tasks, validation, and troubleshooting.

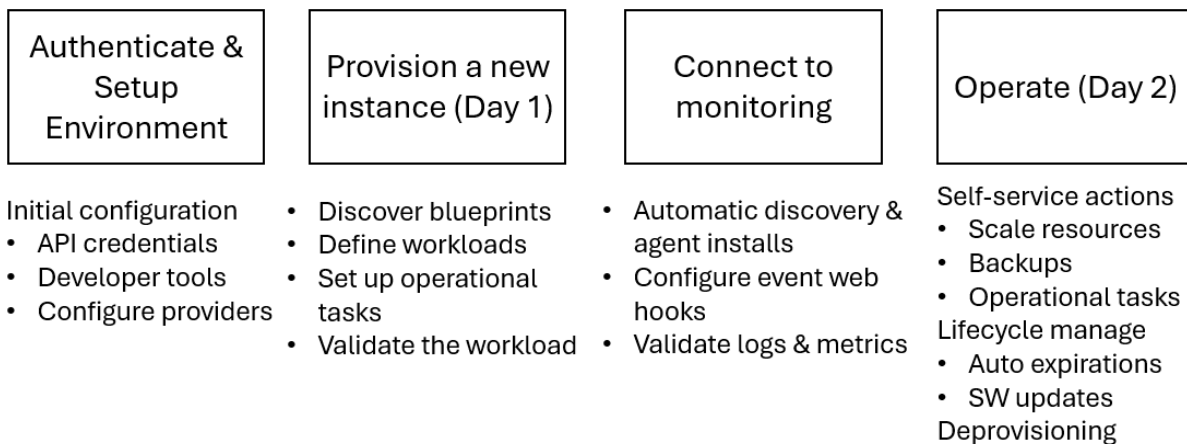
Your product does not need to provide all three interfaces for every pipeline, but each interface it supports should work non-interactively with machine authentication, stable versioning, predictable errors and exit codes, secure secret handling, and useful logs.

Customer Journey

A Golden Path is the product-supported route that enables the most common customer scenario with approved defaults, documented integrations, automated checks, clear ownership, and defined exception handling. It should reduce unnecessary decisions without preventing customers from using supported alternatives when their requirements differ.

Briefly, the onboarding Golden Path moves through these stages:

- purchase and organizational readiness;
- authentication and environment setup;
- provisioning;
- observability;
- lifecycle operations through retirement.



Amycmitchell.substack.com

Purchase, Entitlement, and Organizational Readiness

Before technical setup begins, the customer organization typically completes procurement handoff, license or subscription activation, tenant creation, account ownership assignment, legal and security reviews, data-handling approval, network-access decisions, and internal support ownership.

Authentication and Local Environment Setup

Before provisioning resources, the developer prepares a local or CI/CD environment and obtains an approved machine identity. The organization may also need to issue access, approve network paths, register applications, store secrets, define roles, and document support ownership.

☐ **Configure API credentials and identity**

- ☐ Obtain credentials through the product's supported authentication flow, such as an API token, OAuth 2.0 client credentials, workload identity, or another approved machine-to-machine method.
- ☐ Record the tenant or environment identifier and API base URL. Store API tokens, OAuth client secrets, and other credentials only in the approved secret-management system; obtain webhook callback details only when configuring an event integration.

☐ **Install and configure developer tooling**

- ☐ Install your product's CLI using the installation method documented by the product team, such as an operating-system package manager, language-specific package manager, approved container image, or dedicated installer.

- ☐ Configure the CLI for the target product environment using the product's documented profile or context command. For example, a product might associate a profile named customer-env with its environment URL.

- ☐ **Configure Infrastructure as Code providers**

- ☐ If your product provides or officially supports a Terraform provider, declare the provider and version in the Terraform configuration, then configure authentication through an approved secret-management method.

- ☐ If the product supports Ansible integration, confirm that the control node or CI/CD runner can reach the required endpoints and use the supported inventory plug-in, collection, or modules with approved authentication.

Day-1 Provisioning

The developer provisions a workload from an approved blueprint or Golden Path. The customer organization must define supported configurations, policies, defaults, approvals, ownership, and exception paths, while the product team maintains the templates and integrations that implement them.

- ☐ **Discover approved blueprints and catalog items**

- ☐ Query your product's API or run the example command **yourproduct library instance-types** to review supported workload options and target environments.

- ☐ **Define the workload manifest**

- ☐ Define the workload in IaC or an API request, including the required workload type, environment, capacity, and deployment target.

- ☐ **Add Day-1 operational tasks**

- ☐ Attach approved post-provisioning tasks—such as application deployment, configuration, identity registration, or database initialization—to the provisioning workflow. Use direct host access, including SSH keys, only when the product's security model requires and supports it.

- ☐ **Provision and validate the workload**

- ☐ Submit the provisioning request through the approved IaC workflow, your product's API, or your product's CLI.

- ☐ Verify the provisioning status and capture the non-sensitive connection outputs required by downstream automation, such as resource identifiers,

hostnames, ports, and service endpoints. Keep credentials and secret-bearing connection strings in the approved secret-management system.

Day-2 Observability and Monitoring

Before a workload goes into service, the customer typically validates the controls that apply to its risk and operating model. These may include security, performance, resilience, backup and recovery, data migration, network connectivity, access boundaries, change approval, rollback, support escalation, and operational ownership.

For ongoing management of the product, the developer verifies that telemetry, logs, and alerts reach the approved operational systems. A low-touch experience depends on supported agents or agentless collection, stable event schemas, integration configuration, access controls, retry behavior, and clear ownership when delivery fails.

☐ **Verify automatic discovery and agent installation**

- ☐ Confirm that the supported discovery or registration mechanism added the new instance to monitoring and verify how failed registrations are detected and retried.

- ☐ Verify that required telemetry agents were installed by post-provisioning automation.

☐ **Configure event webhooks and CI/CD feedback**

- ☐ Route monitoring webhook notifications to approved developer channels such as Slack, Microsoft Teams, or PagerDuty.

- ☐ Configure authenticated webhooks or supported event integrations to notify downstream automation when relevant product events occur. Where a CI/CD platform accepts external triggers, validate event filtering, signature verification, replay protection, retry behavior, and idempotent processing.

☐ **Validate log and metric alignment**

- ☐ Confirm that logs and metrics include consistent workload identifiers—such as application name, environment, and owner—through structured log fields, resource attributes, labels, or collection-pipeline enrichment.

Day-2 Lifecycle Operations and Self-Service Maintenance

After production launch, developers and operations teams manage capacity, maintenance, upgrades, backups, recovery, credentials, policy compliance, cost, incidents, and retirement through IaC, your product's API, or your product's CLI.

☐ **Perform Day-2 self-service actions**

- ☐ Scale resources through the declared IaC configuration, your product's supported API, or your product's CLI, then verify that capacity matches the requested state.
- ☐ **Prepare for recovery:** Before a change that may require rollback, use the product's supported recovery method, such as a snapshot, backup, versioned configuration, deployment rollback, or restore point, and verify that the recovery procedure is documented and testable.
- ☐ **Run an approved operational task:** Use a documented automation workflow for routine maintenance, such as restarting a service, clearing a cache, or applying a database change.

☐ **Manage ephemeral environment lifecycles**

- ☐ Apply a **Time-to-Live (TTL)** policy or automatic expiration schedule to sandbox and test environments.
- ☐ Verify that the environment is automatically removed or flagged for review when its expiration criteria are met, according to retention, legal-hold, backup, and dependency policies.

☐ **Tear down and deprovision the environment**

- ☐ Submit the deprovisioning request through the approved IaC workflow, your product's API, or your product's CLI.
- ☐ Confirm that deprovisioning removes or deregisters monitoring integrations and releases product-owned dependent resources, such as IP address allocations and DNS records. Verify that retained data, backups, shared dependencies, and customer-owned resources are handled according to the ownership and retention model and record any cleanup failures for remediation.

Conclusion

This guide is a starting point for creating a Golden Path for your product: a clear, repeatable blueprint that guides customers from purchase through production. Replace these example steps with the work your customers actually perform, then use that journey to define the documentation, integrations, automation, controls, and support needed to make adoption predictable and successful.